

实验1 CPU虚拟化 实验报告

uu-matter543

2025-2026-1

1 调研部分

1.1 可虚拟化架构与不可虚拟化架构

1.1.1 特权指令与敏感指令

- 特权指令：只能在最高特权级别（通常称为**内核态**或**监管态**）下执行的指令。如果在低特权级别（**用户态**）下执行，会引发一个异常（或陷阱），并被操作系统捕获。例如：直接操作硬件、修改内存管理单元（MMU）的设置、关闭中断等。
- 敏感指令：试图改变机器资源配置或会影响整个系统状态的指令。例如：修改页表、改变处理器模式、进行I/O操作等。

1.1.2 可虚拟化架构与不可虚拟化架构

- 一个架构被称为**可虚拟化的**，当且仅当所有敏感指令都是特权指令。
- 一个架构被称为**不可虚拟化的**，当且仅当存在一些敏感指令是非特权指令。

1.2 虚拟化的陷入再模拟

- 陷入：当客户机操作系统执行一条特权指令或访问敏感资源时，由于它运行在非特权级别，硬件会强制引发一个异常（比如通用保护错误），这个异常会被底层的虚拟机监控器捕获。
- 再模拟：VMM 接管控制权后，会分析产生异常的原因，然后完全通过软件模拟这条指令或这次访问想要执行的操作。模拟完成后，VMM 会恢复客户机操作系统的执行，就像这条指令是直接由硬件成功执行了一样。

1.3 Intel VT-x 的特权级与 Hypervisor 陷入再模拟

1.3.1 Intel VT-x 的特权级

Intel VT-x 将 CPU 的运行状态划分为两种模式：

- 根模式：最高特权级别，高于传统的 Ring 0。Hypervisor 运行在此模式下，负责创建、管理、调度和销毁虚拟机。它拥有对物理硬件（如内存、中断控制器、定时器等）的终极控制权。它可以执行所有特权指令，并管理“非根模式”下的所有虚拟机。
- 非根模式：低于根模式。客户操作系统及其上的应用程序运行在此模式下。在客户操作系统的视角下，客户操作系统的内核认为自己运行在 Ring 0，客户应用程序运行在 Ring 3。

1.3.2 Intel VT-x 的特权级与 Hypervisor 陷入再模拟

当在非根模式下运行的客户机触发了某些预设的事件时，CPU 会自动地、硬性地退出非根模式，陷入到根模式。此时，CPU 状态会被自动保存到一块称为 VMCS 的内存结构中，然后跳转到 Hypervisor 预设的处理程序。当 Hypervisor 处理完 VM Exit 事件后，它通过执行一条指令，使 CPU 从根模式进入非根模式，恢复客户机的运行。其中，Hypervisor 可以通过配置 VMCS 来精确控制哪些操作会触发陷入在模拟过程。

Intel VT-x 的非根模式通过引入一个全新的、更高特权的根模式，实现了彻底的权限分离。在此模式下，客户机被禁锢在非根模式中，无法直接接触关键硬件。此外，陷入再模拟过程由硬件自动触发，这比纯软件模拟更迅速、也更可靠，能精确地感知哪些指令和操作是敏感的。在客户机操作系统视角，客户操作系统仍然认为自己运行在 Ring 0–Ring 3 的经典环境中，使得客户操作系统无需修改就能运行。

2 实验目的

1. 学习参考书目，调研虚拟化基础概念，查询相关资料并思考问题
2. 运行代码，修改 IO 部分的陷入再模拟过程，完成 case KVM_EXIT_IO 分支

3 实验步骤

本实验在头歌平台云服务器的虚拟机中进行，步骤如下：

1. 在头歌平台上启动虚拟环境，如图1所示
2. 切换到 virtlab 用户，如图2所示
3. 进入实验目录 ~/labs/cpu_lab 下，如图3所示
4. 使用 vim sample-qemu.c 命令打开源代码文件，如图4所示
5. 输入 I 进入编辑模式，并补充源代码中的 case KVM_EXIT_IO 分支，以将字符打印到标准输出，如图5所示
6. 完成编辑后单击 Esc，输入 :wq 保存并退出，如图6所示
7. 使用 make 编译源代码文件，如图7所示
8. 运行编译得到的目标程序，如图8所示

4 实验分析

4.1 初始化KVM

```
kvm = open("/dev/kvm", O_RDWR | O_CLOEXEC);  
if (kvm == -1)  
    err(1, "/dev/kvm");  
ret = ioctl(kvm, KVM_GET_API_VERSION, NULL);  
if (ret == -1)  
    err(1, "KVM_GET_API_VERSION");
```

```
if (ret != 12)
    errx(1, "KVM_GET_API_VERSION %d, expected 12", ret);
```

打开 KVM 设备文件，并检查是否成功打开，之后检查 API 版本兼容性，即是否为稳定版

4.2 创建虚拟机环境

```
vmfd = ioctl(kvm, KVM_CREATE_VM, (unsigned long)0);
if (vmfd == -1)
    err(1, "KVM_CREATE_VM");
```

创建一个新的虚拟机实例，并检查是否创建成功

4.3 为虚拟机分配内存

```
mem = mmap(NULL, 0x1000, PROT_READ | PROT_WRITE, MAP_SHARED |
    MAP_ANONYMOUS, -1, 0);
if (!mem)
    err(1, "allocating guest memory");
memcpy(mem, code, sizeof(code));
```

分配一页内存，并将汇编代码复制到该页内存中，并检查是否分配成功

```
struct kvm_userspace_memory_region region = {
    .slot = 0,
    .guest_phys_addr = 0x1000,
    .memory_size = 0x1000,
    .userspace_addr = (uint64_t)mem,
};
ret = ioctl(vmfd, KVM_SET_USER_MEMORY_REGION, &region);
if (ret == -1)
    err(1, "KVM_SET_USER_MEMORY_REGION");
```

设置虚拟机内存的起始地址为 0x1000，设置内存大小为 1000F (4KB)，并设置为上述存有汇编代码的内存页，通过 KVM_SET_USER_MEMORY_REGION 将该页内存映射到虚拟机内存，并检查是否映射成功

4.4 汇编代码分析

```
mov $0x3f8, %dx
mov $'H', %al
out %al, (%dx)
mov $'e', %al
out %al, (%dx)
mov $'l', %al
out %al, (%dx)
mov $'l', %al
out %al, (%dx)
mov $'o', %al
```



```
out %al, (%dx)
mov $',', %al
out %al, (%dx)
mov $' ', %al
out %al, (%dx)
mov $'w', %al
out %al, (%dx)
mov $'o', %al
out %al, (%dx)
mov $'r', %al
out %al, (%dx)
mov $'l', %al
out %al, (%dx)
mov $'d', %al
out %al, (%dx)
mov $'!', %al
out %al, (%dx)
mov $'\n', %al
out %al, (%dx)
hlt
```

设置 DX 寄存器为串口端口号 0x03f8，并将 "Hello, world!\n" 的每个字符依次通过 AL 寄存器输出到串口，最后执行 HLT 指令停止 CPU

4.5 创建和配置虚拟CPU

4.5.1 创建 VCPU

```
vcpufd = ioctl(vmfd, KVM_CREATE_VCPU, (unsigned long)0);
if (vcpufd == -1)
    err(1, "KVM_CREATE_VCPU");
```

创建虚拟 CPU，并检查是否创建成功

4.5.2 映射 kvm_run 结构

```
ret = ioctl(kvm, KVM_GET_VCPU_MMAP_SIZE, NULL);
if (ret == -1)
    err(1, "KVM_GET_VCPU_MMAP_SIZE");
mmap_size = ret;
if (mmap_size < sizeof(*run))
    errx(1, "KVM_GET_VCPU_MMAP_SIZE unexpectedly small");
run = mmap(NULL, mmap_size, PROT_READ | PROT_WRITE, MAP_SHARED,
    vcpufd, 0);
if (!run)
    err(1, "mmap vcpu");
```

映射虚拟 CPU 的 kvm_run 结构，以便用户态和内核态之间共享数据，并检查是否映射成功



4.5.3 设置 CS 寄存器

```
ret = ioctl(vcpufd, KVM_GET_SREGS, &sregs);
if (ret == -1)
    err(1, "KVM_GET_SREGS");
sregs.cs.base = 0;
sregs.cs.selector = 0;
ret = ioctl(vcpufd, KVM_SET_SREGS, &sregs);
if (ret == -1)
    err(1, "KVM_SET_SREGS");
```

设置 CS 寄存器指向 0

4.5.4 设置通用寄存器

```
struct kvm_regs regs = {
    .rip = 0x1000,
    .rax = 2,
    .rbx = 2,
    .rflags = 0x2,
};
ret = ioctl(vcpufd, KVM_SET_REGS, &regs);
if (ret == -1)
    err(1, "KVM_SET_REGS");
```

设置通用寄存器，尤其是 IP 寄存器指向 0，即汇编代码的起始位置

4.6 运行虚拟机

```
while (1) {
    ret = ioctl(vcpufd, KVM_RUN, NULL);
    if (ret == -1)
        err(1, "KVM_RUN");
    switch (run->exit_reason) {
        // Dealing with reason
    }
}
```

运行虚拟机，并处理不同的退出原因

```
case KVM_EXIT_IO:
    if (run->io.direction == KVM_EXIT_IO_OUT && run->io.size == 1 &&
        run->io.port == 0x3f8 && run->io.count == 1)
    {
        char c = *(((char *)run) + run->io.data_offset);
        if (c >= 'A' && c <= 'Z') c = c - 'A' + 'a';
        putchar(c);
    }
    else
```



```
errx(1, "unhandled KVM_EXIT_IO");  
break;
```

KVM_EXIT_IO: 处理 I/O 操作（串口输出），当且仅当满足以下所有情况时才输出：

- I/O 操作的方向为输出
- I/O 操作的数据大小为 1 字节
- I/O 操作的端口号为 0x03f8
- I/O 操作的次数为 1 次

满足输出条件的情况下，经过筛选大写字母并将大写字母转变为小写字母的过程，将字符打印到控制台

```
case KVM_EXIT_HLT:  
    puts("KVM_EXIT_HLT");  
    return 0;
```

KVM_EXIT_HLT: 虚拟机执行了 HLT 指令，正常退出

```
case KVM_EXIT_FAIL_ENTRY:  
    errx(1, "KVM_EXIT_FAIL_ENTRY: hardware_entry_failure_reason = 0x%  
        llx", (unsigned long long)run->fail_entry.  
        hardware_entry_failure_reason);
```

KVM_EXIT_FAIL_ENTRY: 处理硬件错误

```
case KVM_EXIT_INTERNAL_ERROR:  
    errx(1, "KVM_EXIT_INTERNAL_ERROR: suberror = 0x%x", run->internal.  
        suberror);
```

KVM_EXIT_INTERNAL_ERROR: 处理内部错误

```
default:  
    errx(1, "exit_reason = 0x%x", run->exit_reason);
```

其他错误情况的处理

在本实验过程中，虚拟机执行了 HLT 指令，正常退出，因此在终端得到了 KVM_EXIT_HLT

5 实验总结

在本实验中，我使用 Linux KVM 接口在用户空间创建和管理虚拟机，进行了完整的虚拟机生命周期管理，了解了如何注入和执行客户机代码，并实现基本的 I/O 模拟（串口输出重定向到标准输出），体验了 KVM API 的基本使用方法，收获颇丰。感谢课程组精心设计的虚拟化实验，也感谢老师和助教一直以来的付出和答疑解惑。

A 完整代码

```
#include <err.h>  
#include <fcntl.h>  
#include <linux/kvm.h>
```



```
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <sys/types.h>

int main(void)
{
    int kvm, vmfd, vcpufd, ret;
    const uint8_t code[] = {
        0xba, 0xf8, 0x03, /* mov $0x3f8, %dx */
        0xb0, 'H',      /* mov $'H', %al */
        0xee,           /* out %al, (%dx) */
        0xb0, 'e',      /* mov $'e', %al */
        0xee,           /* out %al, (%dx) */
        0xb0, 'l',      /* mov $'l', %al */
        0xee,           /* out %al, (%dx) */
        0xb0, 'l',      /* mov $'l', %al */
        0xee,           /* out %al, (%dx) */
        0xb0, 'o',      /* mov $'o', %al */
        0xee,           /* out %al, (%dx) */
        0xb0, ', ',     /* mov $', ', %al */
        0xee,           /* out %al, (%dx) */
        0xb0, ' ',      /* mov $' ', %al */
        0xee,           /* out %al, (%dx) */
        0xb0, 'w',      /* mov $'w', %al */
        0xee,           /* out %al, (%dx) */
        0xb0, 'o',      /* mov $'o', %al */
        0xee,           /* out %al, (%dx) */
        0xb0, 'r',      /* mov $'r', %al */
        0xee,           /* out %al, (%dx) */
        0xb0, 'l',      /* mov $'l', %al */
        0xee,           /* out %al, (%dx) */
        0xb0, 'd',      /* mov $'d', %al */
        0xee,           /* out %al, (%dx) */
        0xb0, '!',      /* mov $'!', %al */
        0xee,           /* out %al, (%dx) */
        0xb0, '\n',     /* mov $'\n', %al */
        0xee,           /* out %al, (%dx) */
        0xf4,          /* hlt */
    };
    uint8_t *mem;
    struct kvm_sregs sregs;
    size_t mmap_size;
```



```
struct kvm_run *run;

kvm = open("/dev/kvm", O_RDWR | O_CLOEXEC);
if (kvm == -1)
    err(1, "/dev/kvm");

/* Make sure we have the stable version of the API */
ret = ioctl(kvm, KVM_GET_API_VERSION, NULL);
if (ret == -1)
    err(1, "KVM_GET_API_VERSION");
if (ret != 12)
    errx(1, "KVM_GET_API_VERSION %d, expected 12", ret);

vmfd = ioctl(kvm, KVM_CREATE_VM, (unsigned long)0);
if (vmfd == -1)
    err(1, "KVM_CREATE_VM");

/* Allocate one aligned page of guest memory to hold the code. */
mem = mmap(NULL, 0x1000, PROT_READ | PROT_WRITE, MAP_SHARED |
    MAP_ANONYMOUS, -1, 0);
if (!mem)
    err(1, "allocating guest memory");
memcpy(mem, code, sizeof(code));

/* Map it to the second page frame (to avoid the real-mode IDT at
0). */
struct kvm_userspace_memory_region region = {
    .slot = 0,
    .guest_phys_addr = 0x1000,
    .memory_size = 0x1000,
    .userspace_addr = (uint64_t)mem,
};
ret = ioctl(vmfd, KVM_SET_USER_MEMORY_REGION, &region);
if (ret == -1)
    err(1, "KVM_SET_USER_MEMORY_REGION");

vcpufd = ioctl(vmfd, KVM_CREATE_VCPU, (unsigned long)0);
if (vcpufd == -1)
    err(1, "KVM_CREATE_VCPU");

/* Map the shared kvm_run structure and following data. */
ret = ioctl(kvm, KVM_GET_VCPU_MMAP_SIZE, NULL);
if (ret == -1)
    err(1, "KVM_GET_VCPU_MMAP_SIZE");
mmap_size = ret;
if (mmap_size < sizeof(*run))
    errx(1, "KVM_GET_VCPU_MMAP_SIZE unexpectedly small");
```



```
run = mmap(NULL, mmap_size, PROT_READ | PROT_WRITE, MAP_SHARED,
vcpufd, 0);
if (!run)
    err(1, "mmap vcpu");

/* Initialize CS to point at 0, via a read-modify-write of sregs.
 */
ret = ioctl(vcpufd, KVM_GET_SREGS, &sregs);
if (ret == -1)
    err(1, "KVM_GET_SREGS");
sregs.cs.base = 0;
sregs.cs.selector = 0;
ret = ioctl(vcpufd, KVM_SET_SREGS, &sregs);
if (ret == -1)
    err(1, "KVM_SET_SREGS");

/* Initialize registers: instruction pointer for our code,
addends, and
 * initial flags required by x86 architecture. */
struct kvm_regs regs = {
    .rip = 0x1000,
    .rax = 2,
    .rbx = 2,
    .rflags = 0x2,
};
ret = ioctl(vcpufd, KVM_SET_REGS, &regs);
if (ret == -1)
    err(1, "KVM_SET_REGS");

/* Repeatedly run code and handle VM exits. */
while (1) {
    ret = ioctl(vcpufd, KVM_RUN, NULL);
    if (ret == -1)
        err(1, "KVM_RUN");
    switch (run->exit_reason) {
    case KVM_EXIT_HLT:
        puts("KVM_EXIT_HLT");
        return 0;
    case KVM_EXIT_IO:
        if (run->io.direction == KVM_EXIT_IO_OUT && run->io.size
            == 1 && run->io.port == 0x3f8 && run->io.count == 1)
        {
            char c = *(((char *)run) + run->io.data_offset);
            if (c >= 'A' && c <= 'Z') c = c - 'A' + 'a';
            putchar(c);
        }
    else
```



```
        errx(1, "unhandled KVM_EXIT_IO");
        break;
    case KVM_EXIT_FAIL_ENTRY:
        errx(1, "KVM_EXIT_FAIL_ENTRY:
            hardware_entry_failure_reason = 0x%llx",
            (unsigned long long)run->fail_entry.
            hardware_entry_failure_reason);
    case KVM_EXIT_INTERNAL_ERROR:
        errx(1, "KVM_EXIT_INTERNAL_ERROR: suberror = 0x%x", run->
            internal.suberror);
    default:
        errx(1, "exit_reason = 0x%x", run->exit_reason);
    }
}
```



```
System load: 0.46142578125    Processes:           123
Usage of /:  49.1% of 39.07GB Users logged in:        0
Memory usage: 10%             IPv4 address for enpls0: 10.0.2.2
Swap usage:  0%

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

  https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

2 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sat Oct  7 23:12:08 2023 from 100.86.166.233
root@virtlab:~#
```

图 1 启动虚拟环境

```
System load: 0.46142578125    Processes:           123
Usage of /:  49.1% of 39.07GB Users logged in:        0
Memory usage: 10%             IPv4 address for enpls0: 10.0.2.2
Swap usage:  0%

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

  https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

2 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sat Oct  7 23:12:08 2023 from 100.86.166.233
root@virtlab:~# su - virtlab
virtlab@virtlab:~$
```

图 2 切换用户



```
Usage of /: 49.1% of 39.07GB  Users logged in: 0
Memory usage: 10%          IPv4 address for enpls0: 10.0.2.2
Swap usage: 0%

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

2 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sat Oct 7 23:12:08 2023 from 100.86.166.233
root@virtlab:~# su - virtlab
virtlab@virtlab:~$ cd ~/labs/cpu_lab
virtlab@virtlab:~/labs/cpu_lab$
```

图 3 进入实验目录

```
if (ret == -1)
    err(1, "KVM_RUN");
switch (run->exit_reason) {
case KVM_EXIT_HLT:
    puts("KVM_EXIT_HLT");
    return 0;
case KVM_EXIT_IO:
    // -----
    //----- START YOUR CODE -----

    // ----- END OF YOUR CODE -----
    // -----
case KVM_EXIT_FAIL_ENTRY:
    errx(1, "KVM_EXIT_FAIL_ENTRY: hardware_entry_failure_reason = 0x%llx",
        (unsigned long long)run->fail_entry.hardware_entry_failure_reason);
case KVM_EXIT_INTERNAL_ERROR:
    errx(1, "KVM_EXIT_INTERNAL_ERROR: suberror = 0x%x", run->internal.suberror);
default:
    errx(1, "exit_reason = 0x%x", run->exit_reason);
}
}
-- INSERT --                                     171,2      Bot
```

图 4 打开源代码文件



```
case KVM_EXIT_HLT:
    puts("KVM_EXIT_HLT");
    return 0;
case KVM_EXIT_IO:
    if (run->io.direction == KVM_EXIT_IO_OUT && run->io.size == 1 && run->io.port == 0x3f8 && run->io.count == 1)
    {
        char c = *((char *)run) + run->io.data_offset;
        if (c >= 'A' && c <= 'Z') c = c - 'A' + 'a';
        putchar(c);
    }
    else
        errx(1, "unhandled KVM_EXIT_IO");
    break;
case KVM_EXIT_FAIL_ENTRY:
    errx(1, "KVM_EXIT_FAIL_ENTRY: hardware_entry_failure_reason = 0x%llx",
        (unsigned long long)run->fail_entry.hardware_entry_failure_reason);
case KVM_EXIT_INTERNAL_ERROR:
    errx(1, "KVM_EXIT_INTERNAL_ERROR: suberror = 0x%x", run->internal.suberror);
default:
    errx(1, "exit_reason = 0x%x", run->exit_reason);
}
}
```

173,1 Bot

图 5 编辑源代码

```
return 0;
case KVM_EXIT_IO:
    if (run->io.direction == KVM_EXIT_IO_OUT && run->io.size == 1 && run->io.port == 0x3f8 && run->io.count == 1)
    {
        char c = *((char *)run) + run->io.data_offset;
        if (c >= 'A' && c <= 'Z') {
            c = c - 'A' + 'a';
        }
        putchar(c);
    }
    else
        errx(1, "unhandled KVM_EXIT_IO");
    break;
case KVM_EXIT_FAIL_ENTRY:
    errx(1, "KVM_EXIT_FAIL_ENTRY: hardware_entry_failure_reason = 0x%llx",
        (unsigned long long)run->fail_entry.hardware_entry_failure_reason);
case KVM_EXIT_INTERNAL_ERROR:
    errx(1, "KVM_EXIT_INTERNAL_ERROR: suberror = 0x%x", run->internal.suberror);
default:
    errx(1, "exit_reason = 0x%x", run->exit_reason);
}
}
```

:wq

图 6 保存源代码



```
* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

2 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sat Oct  7 23:12:08 2023 from 100.86.166.233
root@virtlab:~# su - virtlab
virtlab@virtlab:~$ cd ~/labs/cpu_lab
virtlab@virtlab:~/labs/cpu_lab$ vim sample-qemu.c
virtlab@virtlab:~/labs/cpu_lab$ make
cc sample-qemu.c -o sample-qemu
virtlab@virtlab:~/labs/cpu_lab$
```

图 7 编译源代码

```
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.0-86-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/advantage
Last login: Tue Oct 28 17:45:28 2025 from 100.86.166.192
root@virtlab:~# su - virtlab
virtlab@virtlab:~$ cd labs
virtlab@virtlab:~/labs$ cd cpu_lab
virtlab@virtlab:~/labs/cpu_lab$ sudo ./sample-qemu
hello, world!
KVM_EXIT_HLT
virtlab@virtlab:~/labs/cpu_lab$
```

图 8 运行程序